

WEB TECHNOLOGY RECORD

ALGORITHMS

All 18 Programs | 15 Steps Each | 1 Step = 1 Line

1. Employee Details [HTML]

Aim : To create an HTML webpage that displays employee records in a structured table.

■ START

Step 1 : Declare `<!DOCTYPE html>` to inform the browser that this is an HTML5 document.

Step 2 : Open the `<html>` tag — it is the root element that wraps the entire webpage.

Step 3 : Open `<head>` tag — this section holds metadata not visible on the screen.

Step 4 : Write `<title>Employee List</title>` — this text appears on the browser tab.

Step 5 : Close `</head>` and open `<body>` — all visible content goes inside body.

Step 6 : Write `<h2>Employee Details</h2>` to display the page heading.

Step 7 : Open `<table border='1' cellpadding='5' cellspacing='0'>` to create a bordered table.

Step 8 : Open `<thead>` and write one `<tr>` to begin the table header row.

Step 9 : Add five `<th>` tags: Emp ID, Full Name, Department, Position, Salary — these are column headers.

Step 10 : Close `</tr>` and `</thead>`, then open `<tbody>` to begin the data section.

Step 11 : Add first row: `<tr>` with `<td>` values — 101, John Doe, Engineering, Developer, \$75,000.

Step 12 : Add second row: `<tr>` with `<td>` values — 102, Jane Smith, Marketing, Manager, \$85,000.

Step 13 : Add third row: `<tr>` with `<td>` values — 103, Bob Johnson, Sales, Representative, \$60,000.

Step 14 : Close `</tbody>` and `</table>` to complete the entire table structure.

Step 15 : Close `</body>` and `</html>` — open in browser to see the employee table with 3 rows.

■ STOP

2. Time Table [HTML + CSS]

Aim : To create a weekly college timetable using HTML table with CSS styling, colspan and rowspan.

■ START

Step 1 : Declare `<!DOCTYPE html>`, open `<html>`, `<head>`, and set `<title>Time Table</title>`.

Step 2 : Inside `<style>` tag, set `body background-color: rgb(247, 210, 184)` and `font-family: Arial`.

Step 3 : Style the table: `border-collapse collapse`, `margin auto`, and add `box-shadow`.

Step 4 : Style `th` and `td`: `border 1px solid`, `padding 10px`, `text-align center`.

Step 5 : Define `.highlight` class with background `#fad3fc` (pink) for the Day column cells.

Step 6 : Define `.special` class with background `#bdbcbc` (grey) for LAB, LUNCH, SPORTS periods.

Step 7 : Close `</style>`, open `<body>`, and write `<h1>TIME TABLE</h1>` as the page heading.

Step 8 : Create `<table>` and add the first `<tr>` with `<th>` tags for each period and time slot.

Step 9 : Add Monday row — use `colspan='3'` for WT Lab and `rowspan='6'` for the LUNCH column.

Step 10 : Add Tuesday row — use `colspan='3'` for SE LAB in the morning and SPORTS in the last period.

Step 11 : Add Wednesday row — use `colspan='3'` for LIBRARY in the afternoon periods.

Step 12 : Add Thursday row — use `colspan='3'` for SE LAB in the afternoon.

Step 13 : Add Friday row — use `colspan='3'` for WT LAB in the morning.

Step 14 : Add Saturday row — use `colspan='3'` for SEMINAR and add SPORTS in the last period.

Step 15 : Close `</table>`, `</body>`, `</html>` — open in browser to see a colored, merged timetable.

■ STOP

3. Registration Form [HTML + CSS]

Aim : To create a styled user registration form with input fields, radio buttons, checkboxes, and CSS animation.

■ START

Step 1 : Declare `<!DOCTYPE html>`, set `<meta charset='UTF-8'>` and `<title>Registration</title>`.

Step 2 : Link external CSS file using `<link rel='stylesheet' href='style.css'>` inside head.

Step 3 : In style.css: set body background as `linear-gradient(135deg, #6a11cb, #2575fc)`.

Step 4 : Style .main container: width 380px, white background, border-radius 15px, box-shadow.

Step 5 : Style all input fields: width 100%, padding 10px, border-radius 8px.

Step 6 : Add focus effect on input: border-color blue and box-shadow glow.

Step 7 : Style submit button green (#28a745) and reset button red (#dc3545) with hover scale effect.

Step 8 : Add `@keyframes fadeIn` animation — form fades in from bottom to normal position on load.

Step 9 : In body: write `<h1><u>Registration Form</u></h1>` and open `<div class='main'>` and `<form>`.

Step 10 : Add text inputs for First Name, Last Name, User Name using `type='text'`.

Step 11 : Add password inputs for Password and Confirm Password using `type='password'`.

Step 12 : Add inputs for Address (text), Date of Birth (date), and Phone Number (number).

Step 13 : Add radio buttons for Branch: CSE, IT, ECE, EEE, MECH — all with `name='branch'`.

Step 14 : Add checkboxes for Languages: English, Telugu, Hindi, Kannada, Tamil.

Step 15 : Add Submit and Reset buttons, close `</form>`, `</div>`, `</body>`, `</html>`.

■ STOP

4. CSS Demo — All Three Types [HTML + CSS]

Aim : To demonstrate Inline CSS, Internal CSS, and External CSS in a single HTML program.

■ START

Step 1 : Create style.css — this is the external CSS file.

Step 2 : In style.css: set body { font-family: Arial, sans-serif; }.

Step 3 : In style.css: define .external-box { background-color: lightgreen; padding: 15px; border-radius: 10px; }.

Step 4 : In style.css: define .footer { text-align: center; color: gray; margin-top: 30px; }.

Step 5 : Create the HTML file — declare <!DOCTYPE html> and set <title>Advanced CSS Example</title>.

Step 6 : Link the external CSS: <link rel='stylesheet' href='style.css'> inside <head>.

Step 7 : Write Internal CSS inside <style> tag: .internal-box with background lightblue.

Step 8 : Also in Internal CSS: h2 { color: darkblue; }.

Step 9 : Close </style> and </head>, then open <body>.

Step 10 : Add Inline CSS heading: <h1 style='text-align:center; color:red;'>Welcome to CSS Demo</h1>.

Step 11 : Add Internal CSS section: <div class='internal-box'> with a heading and paragraph.

Step 12 : Add External CSS section: <div class='external-box'> with a heading and paragraph.

Step 13 : Add Inline CSS paragraph: <p style='color:green; text-align:center;'>Inline styling example.</p>.

Step 14 : Add footer: <div class='footer'><p>© 2026 CSS Example Page</p></div>.

Step 15 : Close </body>, </html> — output shows red heading, blue box, green box, green text, grey footer.

■ STOP

5. Factorial of a Number [JavaScript]

Aim : To calculate the factorial of a user-entered number using a JavaScript for loop.

■ START

Step 1 : Declare `<!DOCTYPE html>`, open `<html>`, `<head>`, and set `<title>Factorial</title>`.

Step 2 : Open `<script>` tag inside head — all JavaScript code goes here.

Step 3 : Define the function: `function factorial() { }`

Step 4 : Inside the function, declare variables: `var fact = 1, i;`

Step 5 : Use `prompt()` to get input: `var a = prompt('Enter a number:');`

Step 6 : Start the for loop: `for (i = 1; i <= a; i++) {`

Step 7 : Inside the loop, multiply: `fact = fact * i;`

Step 8 : Close the for loop with `}`.

Step 9 : After the loop, display result: `document.write('Factorial of ' + a + ' is: ' + fact);`

Step 10 : Close the function with `}` and close `</script>` and `</head>`.

Step 11 : Open `<body>` and write `<form>` tag.

Step 12 : Add a button: `<input type='button' value='Factorial' onclick='factorial()'>`.

Step 13 : Close `</form>`, `</body>`, `</html>`.

Step 14 : Example trace — Factorial of 5: $1 \times 1 = 1$, $1 \times 2 = 2$, $2 \times 3 = 6$, $6 \times 4 = 24$, $24 \times 5 = 120$.

Step 15 : Open in browser — click button, enter 5 in prompt, output: Factorial of 5 is: 120.

■ STOP

6. Calculator Program [JavaScript]

Aim : To perform addition, subtraction, multiplication, and division using JavaScript functions.

■ START

Step 1 : Declare `<!DOCTYPE html>`, add meta tags, set `<title>JS Calculator</title>`.

Step 2 : Open `<script>` tag inside head.

Step 3 : Define the function: `function calculate(operation) { }`

Step 4 : Read num1: `let num1 = parseFloat(document.getElementById('num1').value);`

Step 5 : Read num2: `let num2 = parseFloat(document.getElementById('num2').value);`

Step 6 : Declare result and text: `let result, text;`

Step 7 : Validate: `if (isNaN(num1) || isNaN(num2)) { text = 'Enter valid numbers'; }`

Step 8 : Add condition: `else if (operation === 'add') { result = num1 + num2; text = 'Addition Result: ' + result; }`

Step 9 : Add condition: `else if (operation === 'sub') { result = num1 - num2; text = 'Subtraction Result: ' + result; }`

Step 10 : Add condition: `else if (operation === 'multiply') { result = num1 * num2; text = 'Multiplication Result: ' + result; }`

Step 11 : Add divide: `else if (operation === 'divide') { if (num2 !== 0) result = num1 / num2; else text = 'Cannot divide by zero'; }`

Step 12 : Display: `document.getElementById('result').innerHTML = text;` — close function and `</script>`.

Step 13 : In body: add two inputs with `id='num1'` and `id='num2'` of type number.

Step 14 : Add four buttons: Add, Subtract, Multiply, Divide — each calling `calculate()` with its operation name.

Step 15 : Add `<p id='result'></p>` for output, close `</body>`, `</html>`.

■ STOP

7. Area of Rectangle [JavaScript]

Aim : To calculate the area and perimeter of a rectangle using JavaScript prompt() and alert().

■ START

Step 1 : Open <html>, <head>, and set <title>Program-18</title>.

Step 2 : Open <script type='text/javascript'> inside head.

Step 3 : Get length from user: var l = prompt('Enter the length :');

Step 4 : Get breadth from user: var b = prompt('Enter the breadth :');

Step 5 : Calculate area: var area = l * b;

Step 6 : Calculate perimeter: var perimeter = 2 * (l + b);

Step 7 : Display area using alert: alert('The Area of Rectangle = ' + area);

Step 8 : Close </script> and </head>.

Step 9 : Open <body> and immediately close </body> — no visible content on page.

Step 10 : Close </html>.

Step 11 : prompt() shows an input popup — returns the value as a string.

Step 12 : alert() shows an output popup — displays the result message.

Step 13 : The * operator auto-converts string to number for multiplication.

Step 14 : Example: length=5, breadth=3 → area = 15, perimeter = 16.

Step 15 : Open in browser — two prompts appear, then alert shows: The Area of Rectangle = 15.

■ STOP

8. Sorting Array [JavaScript]

Aim : To sort an array of fruit names alphabetically using JavaScript's sort() method.

■ START

Step 1 : Open `<html>`, `<body>` — no head or DOCTYPE needed for this simple program.

Step 2 : Add paragraph: `<p>Click the button to sort the array.</p>`.

Step 3 : Add button: `<button onclick='myFunction()>sort</button>`.

Step 4 : Add display paragraph: `<p id='demo'></p>` — this shows the array.

Step 5 : Open `<script>` tag inside body.

Step 6 : Declare the array: `var fruits = ['Banana', 'Orange', 'Apple', 'Mango'];`

Step 7 : Display original array on load: `document.getElementById('demo').innerHTML = fruits;`

Step 8 : Define the function: `function myFunction() {`

Step 9 : Inside function, call sort: `fruits.sort();`

Step 10 : Update display: `document.getElementById('demo').innerHTML = fruits;`

Step 11 : Close function with `}` and close `</script>`.

Step 12 : Close `</body>` and `</html>`.

Step 13 : `sort()` arranges array elements alphabetically from A to Z.

Step 14 : Original order: Banana, Orange, Apple, Mango.

Step 15 : After sort: Apple, Banana, Mango, Orange — displayed instantly on button click.

■ STOP

9. Number Functions [VBScript]

Aim : To demonstrate VBScript number functions: Rnd, Sqr, Abs, Hex, and IsEmpty.

■ START

Step 1 : Open <html>, <body> tags.

Step 2 : Open <script type='text/vbscript'> — VBScript runs only in Internet Explorer.

Step 3 : Call Randomize() to seed the random number generator using the current time.

Step 4 : Declare variables: Dim rand, Z

Step 5 : Generate random number between 1 and 100: rand = (Int((100-1+1)*Rnd+1))

Step 6 : Display random number: document.write('Today"s random number: ' & rand)

Step 7 : Use Sqr() — display square root: document.write('
The square root of 81 is ' & Sqr(81))

Step 8 : Sqr(81) = 9 because $9 \times 9 = 81$.

Step 9 : Use Abs() — display absolute value: document.write('
Absolute Value of (-1) is ' & Abs(-1))

Step 10 : Abs(-1) = 1 — absolute value removes the negative sign.

Step 11 : Use Hex() — display hexadecimal: document.write('
The hexadecimal value of 34556 is ' & Hex(34556))

Step 12 : Hex(34556) = 86FC — converts decimal to base-16 hexadecimal.

Step 13 : Use IsEmpty() — check uninitialized variable: document.write('
The variable Z has been initialized: ' & IsEmpty(Z))

Step 14 : IsEmpty(Z) = True because Z was declared with Dim but never assigned a value.

Step 15 : Close </script>, </body>, </html> — output shows all 5 results on the page.

■ STOP

10. VBScript Arrays & Methods [VBScript]

Aim : To demonstrate VBScript array creation, indexing, LBound, UBound, and custom Slice function.

■ START

Step 1 : Declare `<!DOCTYPE html>`, open `<html>`, `<head>`, set title.

Step 2 : Open `<script type='text/vbscript'>` inside head.

Step 3 : Define custom function: `Function Slice(aInput, aStart, aEnd)`

Step 4 : Inside `Slice()`: `Dim i, arrReturn()` — declare return array.

Step 5 : `ReDim arrReturn(aEnd - aStart)` — set size of return array dynamically.

Step 6 : Use For loop: `For i = aStart To aEnd` — copy elements to `arrReturn(i - aStart)`.

Step 7 : Set return value: `Slice = arrReturn` — End Function.

Step 8 : Define `Sub DemonstrateArrays()` — this runs when button is clicked.

Step 9 : Declare and create array: `colors = Array('Red','Green','Blue','Yellow','Purple')`

Step 10 : Display full array: `output = output & '<br> Input Array: ' & Join(colors, ',')`

Step 11 : Access element: `output = output & 'Element 0: ' & colors(0) & '<br>'`

Step 12 : Display bounds: `LBound(colors) = 0, UBound(colors) = 4.`

Step 13 : Call `Slice`: `slicedArray = Slice(colors, 1, 3)` — result: Green, Blue, Yellow.

Step 14 : Update page: `document.getElementById('output').innerHTML = output` — End Sub.

Step 15 : In body: add button `onclick='DemonstrateArrays()'` and `<div id='output'></div>`.

■ STOP

11. String Functions [VBScript]

Aim : To demonstrate VBScript string functions: StrReverse, UCase, LCase, and LEN.

■ START

Step 1 : Open <html>, <head> tags.

Step 2 : Open <script type='text/vbscript'>.

Step 3 : Declare string: sometext = 'Hello Everyone!'

Step 4 : Display label: document.write('REVERSE A STRING
')

Step 5 : Use StrReverse(): document.write(StrReverse(sometext)) — output: !enoyrevE olleH

Step 6 : Declare second string: txt = 'Have a nice day!'

Step 7 : Display label: document.write('
UPPERCASE
')

Step 8 : Use UCase(): document.write(UCase(txt)) — output: HAVE A NICE DAY!

Step 9 : Display label: document.write('
 LOWERCASE STRING
')

Step 10 : Use LCase(): document.write(LCase(txt)) — output: have a nice day!

Step 11 : Re-declare: txt = 'Have a nice day!'

Step 12 : Display label: document.write('
LENGTH OF STRING
')

Step 13 : Use LEN(): document.write(LEN(txt)) — output: 16

Step 14 : LEN() counts all characters including spaces and punctuation.

Step 15 : Close </script>, </head>, <body>, </body>, </html>.

■ STOP

12. Registration Form with VBScript Validation [VBScript]

Aim : To validate a registration form and show success/error MsgBox using VBScript.

■ START

Step 1 : Open <html>, <head>, <title>Registration Form</title>.

Step 2 : Write internal CSS: body padding, .container white box, input width 100%, .btn blue background.

Step 3 : Open <script type='text/vbscript'> inside head.

Step 4 : Define: Sub ProcessRegistration()

Step 5 : Declare variables: Dim name, email, pass

Step 6 : Read name: name = document.getElementById('txtName').value

Step 7 : Read email: email = document.getElementById('txtEmail').value

Step 8 : Read pass: pass = document.getElementById('txtPass').value

Step 9 : Check if empty: If name = " Or email = " Or pass = " Then

Step 10 : Show error: MsgBox 'Registration Failed: Please fill in all fields.', 16, 'Error'

Step 11 : Else — show success: MsgBox 'Registration Verified! Welcome, ' & name & '!', 64, 'Success'

Step 12 : Close: End If — End Sub — </script> — </head>.

Step 13 : In body: create <div class='container'> with <h2>Join Us</h2>.

Step 14 : Add 3 input fields: Full Name (id='txtName'), Email (id='txtEmail'), Password (id='txtPass').

Step 15 : Add button: <input type='button' value='Register Now' onclick='ProcessRegistration()'>.

■ STOP

13. Student Grade [PHP]

Aim : To determine and display a student's grade based on marks using PHP if-elseif-else.

■ START

Step 1 : Open PHP block: `<?php`

Step 2 : Declare marks variable: `$marks = 80;`

Step 3 : Write first condition: `if($marks >= 80) {`

Step 4 : Set grade: `$grade = 'S grade';` then close `}`.

Step 5 : Write second condition: `elseif($marks >= 65) {`

Step 6 : Set grade: `$grade = 'A grade';` then close `}`.

Step 7 : Write third condition: `elseif($marks >= 53) {`

Step 8 : Set grade: `$grade = 'E grade';` then close `}`.

Step 9 : Write else block: `else {`

Step 10 : Set grade: `$grade = 'fail';` then close `}`.

Step 11 : Display result: `Echo 'Student grade:' . $grade;`

Step 12 : Close PHP block: `?>`

Step 13 : Trace with `$marks=80`: `if(80>=80)` is TRUE → `$grade = 'S grade'`.

Step 14 : Trace with `$marks=70`: first `if` is FALSE, `elseif(70>=65)` is TRUE → `$grade = 'A grade'`.

Step 15 : Run on server (XAMPP) — output: Student grade:S grade.

■ STOP

14. Area of Rectangle — PHP Function [PHP]

Aim : To calculate rectangle area using a PHP function with default parameter values.

■ START

Step 1 : Open PHP block: `<?php`

Step 2 : Define function with defaults: `function rect_area($length=2, $width=4) {`

Step 3 : Inside function, calculate: `$area = $length * $width;`

Step 4 : Display result: `echo 'area of Rectangle with length' . $length . '&width' . $width . ' is ' . $area;`

Step 5 : Close function with `}`.

Step 6 : Call function without arguments: `rect_area();`

Step 7 : PHP uses default values: `$length=2, $width=4.`

Step 8 : Calculation: `$area = 2 * 4 = 8.`

Step 9 : Output: area of Rectangle with length2&width4 is 8.

Step 10 : Close PHP block: `?>`

Step 11 : In PHP, default parameter values are used when no argument is passed during the call.

Step 12 : The dot (.) operator joins strings and variables in PHP.

Step 13 : To test with custom values: `rect_area(5, 3) → $area = 15.`

Step 14 : `echo` is used to print output to the browser in PHP.

Step 15 : Run on XAMPP server — open `localhost/filename.php` to see the output.

■ STOP

15. Palindrome Number [PHP]

Aim : To check whether a given number is a Palindrome using PHP while loop and digit-reversal logic.

■ START

Step 1 : Open PHP block: <?php

Step 2 : Declare number: \$number = 121;

Step 3 : Store original: \$temp = \$number;

Step 4 : Initialize reversed sum: \$sum = 0;

Step 5 : Start while loop: while(\$number > 0) {

Step 6 : Extract last digit: \$rem = \$number % 10;

Step 7 : Build reversed number: \$sum = \$sum * 10 + \$rem;

Step 8 : Remove last digit: \$number = floor(\$number / 10);

Step 9 : Close while loop: }

Step 10 : Compare: if(\$temp == \$sum) {

Step 11 : Output if palindrome: echo 'Palindrome Number';

Step 12 : Else: } else { echo 'Not Palindrome Number'; }

Step 13 : Close PHP block: ?>

Step 14 : Trace for 121: rem=1,sum=1 → rem=2,sum=12 → rem=1,sum=121 → compare 121==121 → TRUE.

Step 15 : Run on server — output: Palindrome Number.

■ STOP

16. String Functions [PHP]

Aim : To demonstrate PHP string functions strlen() and strrev().

■ START

Step 1 : Open PHP block: <?php

Step 2 : Declare string: \$main_string = 'hello world';

Step 3 : Use strlen() to find length: echo 'length of the string:' . strlen(\$main_string) . '\n';

Step 4 : strlen('hello world') counts all characters including space = 11.

Step 5 : Use strrev() to reverse: echo 'reversed string:' . strrev(\$main_string) . '\n';

Step 6 : strrev('hello world') reverses all characters → 'dlrow olleh'.

Step 7 : Close PHP block: ?>

Step 8 : strlen() counts every character including spaces and punctuation.

Step 9 : strrev() reverses the string character by character from end to start.

Step 10 : dot (.) operator joins string text and function output in echo.

Step 11 : Place .php file in XAMPP htdocs folder.

Step 12 : Start Apache server in XAMPP control panel.

Step 13 : Open browser and go to: <http://localhost/filename.php>

Step 14 : Output line 1: length of the string:11

Step 15 : Output line 2: reversed string:dlrow olleh

■ STOP

17. Student Details — Java Servlet [Java + HTML]

Aim : To collect student details from an HTML form and display them using a Java Servlet.

■ START

Step 1 : Create index.html with a form: `<form action='StudentServlet' method='get'>`.

Step 2 : Add three input fields: Name, Age, Department — each with a name attribute.

Step 3 : Add submit button and close `</form>`, `</body>`, `</html>`.

Step 4 : Create StudentServlet.java — import `java.io.*`, `javax.servlet.*`, `javax.servlet.http.*`

Step 5 : Declare class: `public class StudentServlet extends HttpServlet {`

Step 6 : Override method: `public void doGet(HttpServletRequest request, HttpServletResponse response)`

Step 7 : Set content type: `response.setContentType('text/html');`

Step 8 : Get writer: `PrintWriter out = response.getWriter();`

Step 9 : Read name: `String name = request.getParameter('name');`

Step 10 : Read age: `String age = request.getParameter('age');`

Step 11 : Read dept: `String dept = request.getParameter('dept');`

Step 12 : Write response: `out.println('<h2>Student Details</h2>');`

Step 13 : Print values: `out.println('Name: ' + name + '<br>');` `out.println('Age: ' + age + '<br>');`

Step 14 : Configure web.xml — register servlet with `<servlet-name>`, `<servlet-class>`, and `<url-pattern>`.

Step 15 : Compile, deploy on Tomcat — fill HTML form, click Submit — servlet displays the student details.

■ STOP

18. AJAX Implementation [JavaScript]

Aim : To fetch data from a server asynchronously using XMLHttpRequest without reloading the page.

■ START

Step 1 : Declare `<!DOCTYPE html>`, open `<html>`, `<head>`, set `<title>AJAX Implementation Demo</title>`.

Step 2 : Close `</head>`, open `<body>`, add `<h2>AJAX Data Fetcher</h2>` as heading.

Step 3 : Add trigger button: `<button onclick='loadData()'>Retrieve Data</button>`.

Step 4 : Add display div: `<div id='displayArea'>Data will appear here upon clicking...</div>`.

Step 5 : Open `<script>` tag and define function: `function loadData() {`

Step 6 : Create XHR object: `const xhttp = new XMLHttpRequest();`

Step 7 : Define callback: `xhttp.onload = function() {`

Step 8 : Check response: `if (this.readyState == 4 && this.status == 200) {`

Step 9 : Update page: `document.getElementById('displayArea').innerHTML = this.responseText; };`

Step 10 : Configure request: `xhttp.open('GET', 'https://jsonplaceholder.typicode.com/posts/1', true);`

Step 11 : Send request: `xhttp.send();`

Step 12 : Close function with `}` and close `</script>`, `</body>`, `</html>`.

Step 13 : `readyState == 4` means response is fully received; `status == 200` means success.

Step 14 : `true` in `open()` makes the request asynchronous — page stays active while waiting.

Step 15 : Open in browser, click button — JSON data appears in `displayArea` without page reload.

■ STOP

ALL 18 PROGRAMS — QUICK SUMMARY

#	Program	Language	Key Concept
1	Employee Details	HTML	table, thead, tbody, th, td
2	Time Table	HTML + CSS	colspan, rowspan, CSS classes
3	Registration Form	HTML + CSS	form inputs, radio, checkbox, animation
4	CSS Demo	HTML + CSS	Inline, Internal, External CSS
5	Factorial	JavaScript	for loop, prompt(), document.write()
6	Calculator	JavaScript	function, if-else, parseFloat, innerHTML
7	Area of Rectangle	JavaScript	prompt(), alert(), arithmetic
8	Sorting Array	JavaScript	Array, sort(), onclick, innerHTML
9	Number Functions	VBScript	Rnd, Sqr, Abs, Hex, IsEmpty
10	VBScript Arrays	VBScript	Array(), LBound, UBound, Join, Slice
11	String Functions	VBScript	StrReverse, UCase, LCase, LEN
12	Registration + Valid.	VBScript	getElementById, MsgBox, If-Or validation
13	Student Grade	PHP	if-elseif-else, \$variable, Echo
14	Area of Rectangle	PHP	function with default parameters
15	Palindrome Number	PHP	while loop, % modulus, floor()
16	String Functions	PHP	strlen(), strpos()
17	Student Servlet	Java + HTML	HttpServlet, doGet, getParameter
18	AJAX Demo	JavaScript	XMLHttpRequest, readyState, status 200

Web Technology Record — 15 Steps Per Program — 1 Step = 1 Line